

# Resenha Crítica: Artigo 9 — Hotspot Patterns

Gustavo Pessoa Firmino Duarte

23 de Abril de 2026

## 1 Introdução

O conceito de *hotspot patterns* emerge da interseção entre análise de histórico de versões e métricas de complexidade de código. Ao cruzar dados de frequência de alterações (extraídos de sistemas de controle de versão como Git) com medidas de complexidade estrutural (como a complexidade ciclomática de McCabe), é possível identificar componentes que são simultaneamente complexos e altamente modificados — os chamados *hotspots* do sistema. Esses pontos concentram o maior risco de defeitos e o maior custo de manutenção. A abordagem, popularizada por Adam Tornhill em seus trabalhos sobre análise comportamental de código, transforma o histórico de commits de um repositório em uma ferramenta de diagnóstico arquitetural de grande valor prático.

## 2 Identificação e Padrões de Hotspots

Um *hotspot* é definido pela combinação de dois fatores: alta complexidade interna e alta frequência de alteração. A razão de se combinar esses dois eixos é simples: um módulo complexo que nunca muda não representa risco imediato, pois os desenvolvedores raramente precisam compreendê-lo ou modificá-lo. Da mesma forma, um arquivo simples que muda frequentemente (como um arquivo de configuração) também não é preocupante. O perigo real está nos arquivos que são ao mesmo tempo difíceis de entender e frequentemente tocados pela equipe.

Os padrões que emergem dessa análise incluem: **hotspots de superclasse** — classes que atraem dependências de grande parte do sistema e precisam mudar sempre que qualquer subdomínio evolui; **hotspots de coordenação** — módulos modificados simultaneamente por múltiplos desenvolvedores, indicando acoplamento implícito; e **hotspots de ciclo de vida** — componentes que surgem como hotspots em cada nova fase do produto, sinalizando que a estrutura arquitetural não acompanhou o crescimento do sistema.

A análise de acoplamento temporal (*temporal coupling*) é especialmente reveladora: quando dois arquivos são alterados juntos consistentemente em commits distintos, há um acoplamento implícito entre eles que não aparece na análise estática do código. Esse padrão frequentemente indica que uma abstração está faltando ou que uma fronteira arquitetural foi traçada incorretamente.

### 3 Conexão com a Prática Profissional

A ideia de usar o histórico de commits como fonte de informação arquitetural é algo que nunca havia considerado antes de estudar esse tema. No meu MVP de e-commerce, o repositório Git registrou fielmente os pontos de maior turbulência: o arquivo de rotas principal e o serviço de produtos foram modificados em quase todos os commits, enquanto utilitários e componentes de UI menores permaneceram estáveis. Retrospectivamente, esse é um sinal clássico de que o serviço de produtos acumulava muitas responsabilidades — era simultaneamente o ponto de acesso ao banco de dados, o lugar de transformação de dados e o ponto de orquestração de regras de negócio.

Na ArcelorMittal, os sistemas legados mais críticos do suporte eram exatamente aqueles que, intuitivamente, a equipe sabia serem os mais “perigosos” de tocar. A abordagem de hotspot patterns oferece uma forma objetiva e mensurável de confirmar essa intuição e de priorizar refatorações com base em evidências quantitativas em vez de opiniões subjetivas.

### 4 Conclusão

A metodologia de análise de hotspot patterns representa uma ponte valiosa entre a teoria de arquitetura de software e a realidade empírica dos repositórios de código. Ao usar dados concretos de histórico de versões, ela torna o diagnóstico arquitetural acessível a equipes que não têm tempo ou recursos para análises formais extensas. Além disso, ao revelar acoplamentos implícitos e pontos de risco concentrado, a abordagem orienta investimentos de refatoração onde o retorno é maior. Em um cenário em que dívida técnica acumulada torna sistemas cada vez mais custosos de manter, a capacidade de identificar objetivamente os piores hotspots é uma vantagem competitiva real para equipes de engenharia.